2025 - 2026

ESIEE PARIS

Intermédiaire #1

La communauté

Rapport Projet-Zuul

ILYAS DAMIRI

GROUPE : 3

Table des matières

1 Introduction au projet Zuul « La communauté » 3
2 Réponses aux exercices 6
Exercice 7.4 - Création des lieux 6
Exercice 7.5 - Factorisation de l'affichage 7
Exercice 7.6 — Encapsulation des sorties 7
Exercice 7.7 - Méthode getExitString() 8
Exercice 7.8 - Utilisation HashMap pour les sorties 9
Exercice 7.8.1 - Déplacements (up – down) 10
Exercice 7.9 — Utilisation de keySet() et boucle for-each 11
Exercice 7.10 - la méthode getExitString() (optionnel) 11
Exercice 7.11 - Ajouts de getLongDescription() 12
Exercice 7.12 - Diagramme objet (optionnel) 12
Exercice 7.13 - Effet de la commande go (optionnel) 13
Exercice 7.14 - Ajout de la commande look 13
Exercice 7.15 - Ajouter la commande eat 14
Exercice 7.16 - Améliorer de la commandes (help) 15
Exercice 7.17 - Limite de processCommand() (optionnel) 17
Exercice 7.18 - Séparation des responsabilités 17
Exercice 7.18 .1 - Comparaison avec zuul-better.jar 18
Exercice 7.18.2 - StringBuilder (OPTIONNEL) 19
Exercice 7.18.3 - Images des salles 19
Exercice 7.18.4 - Titre et description du jeu 19
Exercice 7.18.5 - Gestion globale des salles (OPTIONNEL) 20

Lien du Github :

https://github.com/ilousha06/projet-zuul/commits?author=ilousha06

1 Introduction au projet Zuul « La communauté »

Auteur :

- Nom : **Damiri Ilyas**
- Groupe : **3**

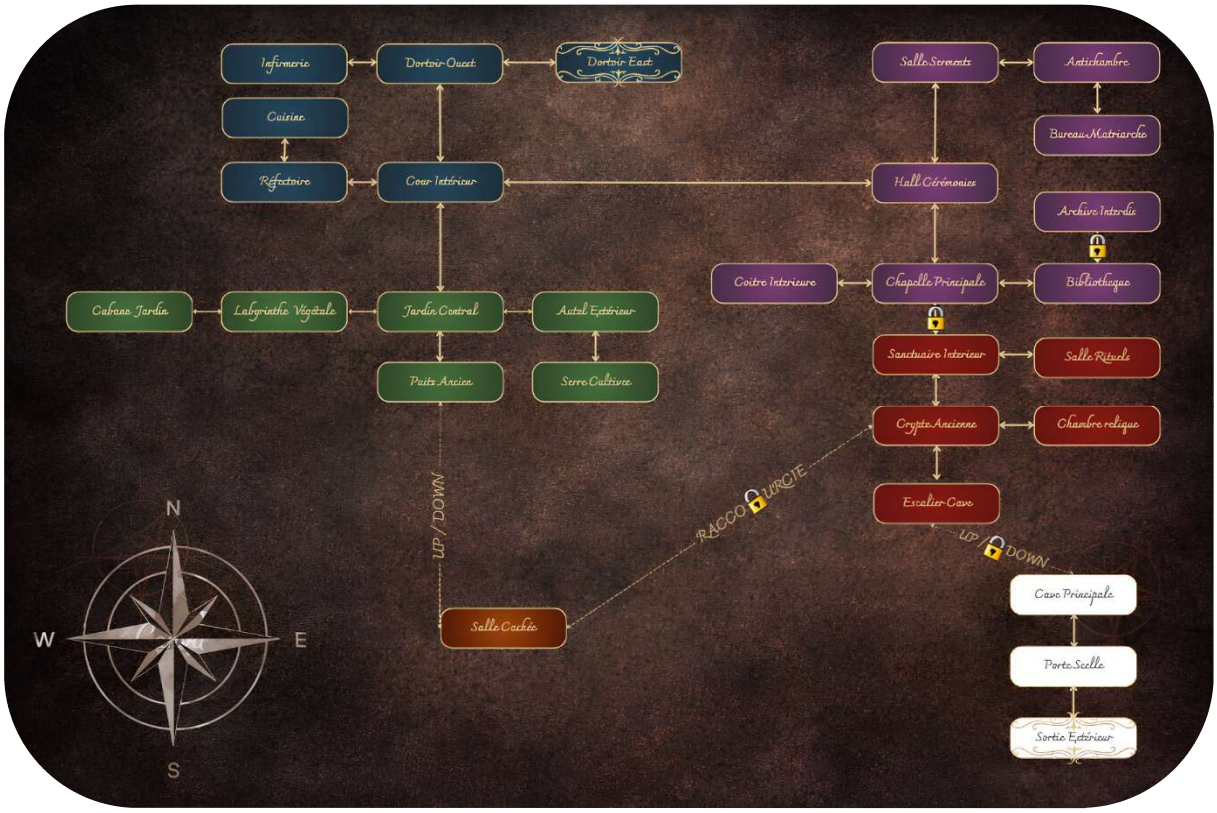
Phrase-thème :

Une jeune novice doit découvrir les secrets d'une communauté religieuse dirigée par une matriarche et parvenir à s'en échapper.

Résumé du scénario :

Le joueur incarne une jeune novice vivant dans une communauté religieuse fermée.
Au fil de ses déplacements, elle découvre que cette communauté est en réalité une secte dirigée par une matriarche, pratiquant des rituels secrets.
Le joueur doit explorer les différents lieux, découvrir des indices et trouver un moyen de quitter définitivement la communauté sans être repéré.

Plan du jeu :



Scénario détaillé :

La jeune novice vit dans une communauté religieuse fermée dirigée par une matriarche. Le jeu débute dans le dortoir, lieu de vie quotidien où tout semble normal. En explorant progressivement les différents espaces de la communauté - la cour intérieure, les bâtiments religieux, les jardins et les zones interdites - le joueur découvre des indices inquiétants.

Certains lieux cachés ne sont accessibles que par des passages discrets ou des déplacements verticaux, comme le puits ancien menant à une salle secrète. Ces explorations permettent à la novice de comprendre que la communauté est en réalité une secte pratiquant des rituels occultes.

Une fois cette vérité découverte, la novice comprend qu'elle est en danger et doit poursuivre son exploration avec prudence afin de trouver la sortie extérieure et s'échapper avant d'être repérée par la matriarche.

Détail des lieux, items, personnages :

Lieux :

Le jeu se déroule dans une communauté religieuse fermée composée de plusieurs zones distinctes :

- **Dortoirs** : lieux de vie de la novice, point de départ du jeu.
- **Cour intérieure** : zone centrale reliant les principaux bâtiments.
- **Réfectoire et cuisine** : lieux de la vie quotidienne.
- **Jardin central** : espace extérieur donnant accès à plusieurs zones annexes.
- **Puits ancien** : permet un déplacement vertical vers une salle cachée.
- **Salle cachée** : lieu secret révélant des indices importants.
- **Chapelle et bâtiments religieux** : zones liées aux rites officiels.
- **Zone occulte** (sanctuaire, crypte) : lieux interdits révélant la véritable nature de la communauté.
- **Cave et souterrains** : dernière étape avant la sortie.
- **Sortie extérieure** : objectif final du jeu.

Items :

À ce stade du projet, aucun objet n'est encore implémenté dans le jeu. Des objets seront ajoutés ultérieurement afin de permettre certaines actions du joueur, comme débloquer des lieux fermés, révéler des indices ou faciliter la progression dans l'environnement. Ces objets pourront par exemple prendre la forme de clés, de documents ou d'éléments liés aux rituels de la communauté.

Personnages :

- **La jeune novice :** personnage incarné par le joueur. Elle découvre progressivement les règles strictes de la communauté et cherche à comprendre la vérité afin de s'en échapper.
- **La matriarche :** dirigeante autoritaire de la communauté. Elle contrôle les lieux, impose les rituels et représente l'antagoniste principale du scénario.
- **Les autres sœurs :** personnages non jouables présents dans différents lieux. Elles renforcent l'ambiance de la communauté fermée et participent à la narration par leurs comportements et leurs silences.
- **La sœur ancienne :** membre expérimentée de la communauté. Elle connaît les règles et les secrets du lieu, et peut parfois laisser transparaître des doutes ou des avertissements implicites.
- **La sœur novice disparue :** ancienne novice dont la disparition soulève des interrogations et sert de fil conducteur narratif.
- **Le gardien du sanctuaire :** figure silencieuse chargée de surveiller les zones interdites et de contrôler l'accès aux lieux sensibles.
- **L'érudite de la bibliothèque :** sœur responsable des archives et des textes sacrés. Elle détient des informations importantes sur l'histoire et les rituels de la communauté.

2 Réponses aux exercices

Exercice 7.4 - Création des lieux

La méthode createRooms() a été modifiée afin de créer l'ensemble des salles constituant le monde du jeu.
Chaque salle est instanciée avec une description textuelle, puis reliée aux autres salles par des sorties correspondant aux directions possibles.
Ces connexions permettent au joueur de se déplacer d'un lieu à un autre.
Enfin, la salle de départ est définie afin d'initialiser la position du joueur au lancement du jeu.

```java
private void createRooms()  1 usage
{
    Room vDortoirEst        = new Room( pDescripion: "dans le dortoir est reserve aux soeurs");
    Room vDortoirOuest      = new Room( pDescripion: "dans le dortoir ouest plus silencieux");
    Room vRefectoire        = new Room( pDescripion: "dans le refectoire commun");
    Room vCuisine           = new Room( pDescripion: "dans la cuisine ou les repas sont prepares");
    Room vInfirmerie        = new Room( pDescripion: "dans l infirmerie aux lumieres froides");
    Room vHallCeremonies    = new Room( pDescripion: "dans le hall des ceremonies");
    Room vCourIntern        = new Room( pDescripion: "dans la cour interieure entouree de murs");
    Room vJardinCentral     = new Room( pDescripion: "dans le jardin central sombre");
    // ...

    // HUB CENTRAL
    vCourIntern.setExit( direction: "north", vDortoirOuest);
    vCourIntern.setExit( direction: "south", vJardinCentral);
    vCourIntern.setExit( direction: "east",  vHallCeremonies);
    vCourIntern.setExit( direction: "west",  vRefectoire);

    vDortoirEst.setExit( direction: "west",  vDortoirOuest);
    vDortoirOuest.setExit( direction: "south", vCourIntern);
    vDortoirOuest.setExit( direction: "west", vInfirmerie);
    vDortoirOuest.setExit( direction: "east", vDortoirEst);

    vInfirmerie.setExit( direction: "east", vDortoirOuest);

    // ...

    this.aCurrentRoom = vDortoirEst;
}
```

SUITE DES ROOM

SUITE DES CHEMINS

Exercice 7.5 - Factorisation de l'affichage

Afin d'éviter la duplication de code et d'afficher la description de la pièce courante ainsi que les sorties disponibles, une procédure printLocationInfo() a été créée. Le code nécessaire à cet affichage était déjà présent dans la méthode goRoom() et a simplement été déplacé dans cette nouvelle procédure. Celle-ci est ensuite appelée dans printWelcome() et dans goRoom().

La procédure printLocationInfo() permet ainsi de centraliser l'affichage et d'améliorer la lisibilité du code.

```
private void printLocationInfo(){  2 usages
    System.out.println("You are " + this.aCurrentRoom.getDescription());
    System.out.print("Exits : ");
    if (this.aCurrentRoom.aNorthExit != null){
        System.out.print("north ");
    }
    if (this.aCurrentRoom.aSouthExit != null){
        System.out.print("south ");
    }
    if (this.aCurrentRoom.aEastExit != null){
        System.out.print("east ");
    }
    if (this.aCurrentRoom.aWestExit != null){
        System.out.print("west ");
    }
}
```

Exercice 7.6 — Encapsulation des sorties

Un getter getExit(String direction) est ajouté dans la classe Room pour récupérer la salle située dans la direction indiquée à partir du lieu actuel.
La méthode renvoie null si la direction demandée n'existe pas ou si aucune salle n'est accessible dans cette direction.

Dans Room :

```java
public Room getExit(String direction)  1 usage
{
    return switch (direction) {
        case "north" -> this.aNorthExit;
        case "south" -> this.aSouthExit;
        case "east" -> this.aEastExit;
        case "west" -> this.aWestExit;
        default -> null;
    };

}
```

Dans Game (dans goRoom) :

```java
private void goRoom(Command pCommand)  1 usage
{
    if (!pCommand.hasSecondWord()) {
        System.out.println("Go where ?");
        return;
    }

    String vDirection = pCommand.getSecondWord();
    Room vNextRoom = null;

    vNextRoom = this.aCurrentRoom.getExit(vDirection);

    if (vNextRoom == null) {
        System.out.println("There is no door !");
        return;
    }

    this.aCurrentRoom = vNextRoom;
    printLocationInfo();
}
```

Exercice 7.7 - Méthode getExitString()

La méthode getExitString() est déplacée dans la classe Room afin que chaque salle fournisse elle-même la liste de ses sorties.

La classe Game ne s'occupe plus de construire cette information et se contente désormais de l'afficher.

Cette modification permet de mieux répartir les responsabilités entre les classes et de réduire le couplage entre Game et Room.

Dans la classe Room :

```
public String getExitString()  2 usages
{
    StringBuilder vExits = new StringBuilder();

    if (this.aNorthExit != null) vExits.append("north ");
    if (this.aEastExit  != null) vExits.append("east ");
    if (this.aSouthExit != null) vExits.append("south ");
    if (this.aWestExit  != null) vExits.append("west ");

    return vExits.toString();
}
```

Dans la classe Game (**appel de la méthode**) :

```
private void printLocationInfo()  2 usages
{
    System.out.println(this.aCurrentRoom.getDescription() + '\n'
            + "Exits: " + this.aCurrentRoom.getExitString());
}
```

Exercice 7.8 - Utilisation HashMap pour les sorties

Tout d'abord, une HashMap nommée aExits est créée afin de stocker les différentes sorties d'une salle en associant chaque direction (north, south, east, west) à l'objet Room correspondant. Cela permet d'accéder directement à une salle à partir de sa direction.

Le constructeur de la classe Room est ensuite modifié pour initialiser cette HashMap, et la méthode getExit() est adaptée afin de rechercher la salle associée à une direction donnée dans cette structure.

La méthode setExits() est remplacée par une nouvelle méthode setExit(String direction, Room neighbor), permettant d'ajouter une sortie pour une direction précise.

Enfin, la méthode createRooms() de la classe Game est mise à jour afin de prendre en compte ces modifications et de définir les connexions entre les salles à l'aide de la nouvelle méthode setExit().

- Déclaration de l'attribut dans Room :

```
private final HashMap<String, Room> aExits;  4 usages
```

- Initialisation dans le constructeur :

```
public Room(final String pDescripion)  29 usages
{
    this.aDescription = pDescripion;
    this.aExits = new HashMap<>();
}
```

- Méthode d'ajout d'une sortie :

```
public void setExit(String direction, Room neighbor)  56 usages
{
    this.aExits.put(direction, neighbor);
}
```

- Méthode d'accès à une sortie :

```
public Room getExit(final String direction)  1 usage
{
    return this.aExits.get(direction);
}
```

Exercice 7.8.1 - Déplacements (up – down)

Certaines salles du jeu sont désormais situées à des niveaux différents.

Le puits ancien permet de descendre vers une salle cachée située en contrebas, et il est possible de remonter depuis cette salle par le même passage.

De plus, l'accès aux zones souterraines (**crypte et cave**) se fait par des escaliers, introduisant ainsi des déplacements verticaux (**up et down**) dans le jeu.

Cette modification enrichit le plan des lieux et permet d'accéder à des zones cachées du scénario.

Exercice 7.9 — Utilisation de keySet() et boucle for-each

À la suite de la création de la HashMap aExits, la méthode getExitString() doit être réécrite, car l'ancienne version ne permet plus de manipuler correctement les sorties.

La méthode keySet() de la HashMap est utilisée afin d'obtenir l'ensemble des directions possibles depuis une salle. Cette méthode retourne un Set<String> contenant les clés de la HashMap, c'est-à-dire les directions (north, south, up, etc.).

Pour pouvoir manipuler cet ensemble, la classe Set doit être importée dans la classe Room.

Une boucle de type *for-each* permet ensuite de parcourir cet ensemble et de construire une chaîne de caractères listant toutes les sorties disponibles pour un lieu. Pour la rend indépendante du nombre de directions et facilite l'ajout de nouvelles sorties.

```
public String getExitString()  2 usages
{
    StringBuilder vExit = new StringBuilder("Exits:");

    for (String direction : this.aExits.keySet()) {
        vExit.append(" ").append(direction);
    }

    return vExit.toString();
}
```

Exercice 7.10 - la méthode getExitString() (optionnel)

La méthode getExitString() permet d'afficher les directions possibles pour sortir d'une salle. Elle utilise la méthode keySet() de la HashMap des sorties afin de parcourir dynamiquement toutes les directions disponibles. Chaque direction est ajoutée à une chaîne de caractères, qui est ensuite retournée et affichée au joueur. Cette approche évite d'écrire les directions en dur dans le code et rend le programme plus flexible pour l'ajout de nouvelles directions.

Et JavaDoc compléter et générer .

Exercice 7.11 - Ajouts de getLongDescription()

On ajoute la méthode getLongDescription() dans la classe Room. Cette méthode permet de regrouper la description de la salle et les sorties possibles dans une seule chaîne de caractères.

La méthode printLocationInfo() de la classe Game utilise alors directement getLongDescription() pour afficher les informations de la salle courante. Cela évite de dupliquer le code dans la classe Game et rend le programme plus simple à maintenir.

Dans Room :

```
public String getLongDescription()  1 usage
{
    return "Vous êtes " +  this.aDescription + ".\n" + getExitString();
}
```

Dans Game :

```
private void printLocationInfo()  2 usages
{
    System.out.println(aCurrentRoom.getLongDescription());
}
```

Exercice 7.12 - Diagramme objet (optionnel)

Lors du lancement du programme, plusieurs objets Java sont créés (Game,Room,Parser...). Un objet Game est instancié et constitue le point d'entrée du jeu. Cet objet crée ensuite un Parser, chargé de lire et d'interpréter les commandes du joueur, ainsi qu'un objet CommandWords qui contient la liste des commandes reconnues.

La méthode createRooms() de la classe Game crée plusieurs instances de la classe Room, chacune représentant une salle du jeu. Ces salles sont reliées entre elles par des références stockées dans une structure de type HashMap, ce qui permet de définir les sorties possibles entre les salles. L'objet Game conserve une référence vers la salle courante, correspondant à la position actuelle du joueur.

Exercice 7.13 - Effet de la commande go (optionnel)

Lors de l'exécution d'un « **go** », le diagramme objet ne change pas dans sa structure. Aucun nouvel objet n'est créé et aucun objet n'est supprimé. Les différentes instances de Room existent déjà et restent reliées entre elles de la même manière.
La seule modification concerne la référence aCurrentRoom de la classe Game, qui pointe vers une autre instance de Room correspondant à la nouvelle position du joueur.

Exercice 7.14 - Ajout de la commande look

Une nouvelle commande look a été ajoutée au jeu. Cette commande permet au joueur d'afficher à nouveau la description de la salle courante ainsi que les sorties disponibles, sans se déplacer. L'affichage repose sur la méthode getLongDescription() de la classe Room, ce qui évite la duplication de code.

En option, une vérification a été ajoutée afin de détecter la présence d'un second mot. Si la commande look est suivie d'un paramètre, **un message d'erreur** est affiché indiquant que cette fonctionnalité n'est pas encore disponible. Cette amélioration prépare les exercices optionnels suivants.

Dans CommandWords (**Ajoutée ''look'' dans le tableau**) :

```java
public class CommandWords  2 usages
{
    private final String[] aValidCommands = {"go", "help", "quit", "look"};  1 usage

    public boolean isCommand(final String pString)  1 usage
    {
        for(String vCommand : aValidCommands) {
            if(vCommand.equals(pString)) {
                return true;
            }
        }
        return false;
    }
}
```

Dans Game.processCommand() :

```
String vCommandWord = pCommand.getCommandWord();

switch (vCommandWord) {
    case "help" -> {
        this.printHelp();
        return false;
    }
    case "go" -> {
        this.goRoom(pCommand);
        return false;
    }
    case "look" -> {
        this.look(pCommand);
        return false;
    }
    case "quit" -> {
        return this.quit(pCommand);
    }
```

Ajoutée une nouvelle procédure :

```
private void look (final Command pCommand)  1 usage
{
    if (pCommand.hasSecondWord())
    {
        System.out.println("I don't know how to look at something in particular yet.");
        return;
    }
    System.out.println(this.aCurrentRoom.getLongDescription());
}
```

Exercice 7.15 - Ajouter la commande eat

De la même manière que nous avons ajouté la commande look, nous ajoutons la commande eat. On obtient alors un nouveau tableau de commandes valides incluant cette commande. Le code associé permet simplement d'afficher un message indiquant que le joueur a mangé et qu'il n'a plus faim. Cette commande est volontairement simple et pourra être améliorée ultérieurement pour gérer réellement la faim du joueur.

Dans CommandWords (**Ajoutée ''eat'' dans le tableau**) :

```
public class CommandWords  2 usages
{
    private final String[] aValidCommands = {"go", "help", "quit", "look", "eat"};  1 usage

    public boolean isCommand(final String pString)  1 usage
    {
        for(String vCommand : aValidCommands) {
            if(vCommand.equals(pString)) {
                return true;
            }
        }
        return false;
    }
}
```

Dans Game.processCommand() :

```
        }
        case "look" -> {
            this.look(pCommand);
            return false;
        }
        case "eat" -> {
            this.eat();
            return false;
        }
        case "quit" -> {
            return this.quit(pCommand);
        }
    }
    return false;
```

Ajoutée une nouvelle procédure :

```
private void eat()  1 usage
{
    System.out.println("You have eaten now and you are not hungry any more.");
}
```

Exercice 7.16 - Améliorer de la commandes (help)

Après l'ajout des commandes look et eat, la méthode help() n'est plus adaptée car de nouvelles commandes peuvent être utilisées. Pour corriger cela, on crée la méthode showAll() dans la classe CommandWords, qui permet d'afficher toutes les commandes valides.

Ensuite, afin d'éviter un couplage direct entre les classes, on crée la méthode showCommands() dans la classe Parser, qui sert de relais entre Game et CommandWords. La méthode printHelp() de la classe Game est alors modifiée pour utiliser ce nouveau mécanisme.

Dans CommandWords :

```
public String getCommandList()  1 usage
{
    StringBuilder vCommands = new StringBuilder();

    for(String vCommand : aValidCommands) {
        vCommands.append(vCommand).append(" ");
    }

    return vCommands.toString();
}
```

Dans Parser :

```
public CommandWords getCommandWords()  1 usage
{
    return this.aCommandWords;
}
```

Dans Game.printHelp() :

```
private void printHelp()  1 usage
{
    System.out.println("You are lost. You are alone.");
    System.out.println("You wander around at the university.");
    System.out.println("Your command words are:");
    System.out.println(this.aParser.getCommandWords().getCommandList());
}
```

Exercice 7.17 - Limite de processCommand() (optionnel)

Lorsque l'on souhaite ajouter une nouvelle commande au jeu, il est nécessaire de modifier la classe Game. En effet, la méthode processCommand() contient la logique permettant de reconnaître les mots-clés des commandes et d'exécuter les actions associées.
Ainsi, chaque nouvelle commande implique l'ajout d'un nouveau cas dans cette méthode, ce qui montre une limite de la conception actuelle et un fort couplage entre la classe Game et les commandes disponibles.

Exercice 7.18 - Séparation des responsabilités

Afin de limiter le couplage implicite entre les classes, la classe CommandWords ne doit plus afficher directement les commandes disponibles. Pour cela, une nouvelle méthode getCommandList() est créée. Elle fonctionne sur le même principe que l'ancienne méthode showAll(), mais au lieu d'afficher les commandes, elle retourne une chaîne de caractères contenant l'ensemble des commandes du tableau aValidCommands.

Ensuite, la classe Parser est modifiée pour jouer un rôle d'intermédiaire. Elle fournit une méthode permettant de récupérer la liste des commandes depuis CommandWords, sans les afficher.

Enfin, la méthode printHelp() de la classe Game est modifiée afin d'afficher en console les commandes valides en utilisant uniquement les méthodes du Parser. Ainsi, l'affichage est centralisé dans la classe Game, tandis que la production et la transmission de l'information sont correctement séparées.

Dans class CommandWords :

```java
    public String getCommandList()  1 usage
    {
        StringBuilder vCommands = new StringBuilder();

        for(String vCommand : aValidCommands) {
            vCommands.append(vCommand).append(" ");
        }

        return vCommands.toString();
    }

}
```

Dans class Parser :

```java
    public String getCommandList()  1 usage
    {
        return this.aCommandWords.getCommandList();
    }
```

Dans class Game :

```java
    private void printHelp()  1 usage
    {
        System.out.println("You are lost. You are alone.");
        System.out.println("You wander around at the university.");
        System.out.println("Your command words are:");
        System.out.println(this.aParser.getCommandList());
    }
```

Exercice 7.18 .1 - Comparaison avec zuul-better.jar

Après comparaison avec le projet zuul-better.jar, aucune modification majeure n'a été nécessaire.
La gestion des sorties, des descriptions des salles et l'utilisation des boucles for-each sont déjà conformes.
Le fonctionnement du jeu reste identique et aucune régression n'a été introduite.
Le projet compile sans avertissement.

Exercice 7.18.2 - StringBuilder (OPTIONNEL)

L'utilisation de StringBuilder permet de construire efficacement des chaînes de caractères lorsqu'elles sont modifiées plusieurs fois, notamment dans des boucles. Contrairement à String, qui est immutable, StringBuilder est mutable et évite la création inutile de nouveaux objets.
Il est particulièrement adapté à des méthodes comme Room.getExitString() ou getCommandList().

Exercice 7.18.3 - Images des salles

Pour améliorer l'immersion du joueur, chaque salle du jeu aura une image qui représente son ambiance (chapelle, jardin, cave, dortoir, etc.).

Toutes les images auront le même format (par exemple 800×600) pour garder une présentation propre. Elles peuvent être des photos ou des illustrations, adaptées à l'univers sombre du jeu.

L'ajout des images se fera petit à petit et ne bloque pas la suite du projet.

Exercice 7.18.4 - Titre et description du jeu

Le titre retenu est « **La Communauté** ».
Ce titre est intégré au message de bienvenue du jeu, apparaît dans le rapport et pourra également être utilisé sur la page web de présentation du projet.

```
private void printWelcome()  1 usage
{
    System.out.println();
    System.out.println("==================================");
    System.out.println("        LA COMMUNAUTÉ             ");
    System.out.println("==================================");
    System.out.println("Une communauté religieuse fermée.");
    System.out.println("Des secrets. Des rituels. Une seule issue.");
    System.out.println("Type 'help' if you need help.");
    System.out.println();
    printLocationInfo();
}
```

Exercice 7.18.5 - Gestion globale des salles (OPTIONNEL)

À ce stade du projet, toutes les salles du jeu sont correctement créées et reliées entre elles dans la méthode createRooms(). Le jeu est donc pleinement fonctionnel et la navigation entre les différentes zones est possible.

Cependant, les objets Room ne sont pas encore regroupés dans une structure de données globale (tableau, ArrayList ou HashMap) permettant d'y accéder depuis n'importe quelle méthode ou classe. L'accès aux salles reste volontairement limité au contexte de leur création.

L'énoncé indiquant que cet exercice est **optionnel tant que l'exercice 7.46 n'est pas atteint**, cette centralisation des salles n'a pas encore été mise en place. Elle pourra être ajoutée ultérieurement si des fonctionnalités avancées nécessitent un accès global aux salles.